

智能晨检仪 对外接口文档

本文档面向客户平台开发人员，说明客户系统调用晨检仪，以及晨检仪主动调用客户平台时的接口约定。文档版本：v2.0 | 更新日期：2026-07-28

目录

- 1. 基础信息
- 2. 设备端 REST API — 客户系统调用设备
- 3. 平台接入接口 — 设备调用客户平台
- 4. 调用示例
- 5. 注意事项
- 6. 更新日志

一、基础信息

项目	说明
设备端基础地址	http://{设备IP}:8080
平台端基础地址	设备「设置 → 平台接入」中配置的平台地址
协议	HTTP / HTTPS；设备端当前固定提供 HTTP
数据格式	JSON；图片下载接口返回二进制数据
字符编码	UTF-8
时间戳	Unix 时间戳，单位毫秒

接口分为两个方向：

- 1. 设备端 REST API：客户系统主动访问晨检仪，使用 JWT Bearer Token 认证。
- 2. 平台接入接口：晨检仪主动访问客户平台，使用 api-key 请求头认证。

请求限制

接口	限制
员工批量导入	单次 1~100 条
用户账号同步	单次最多 100 条
设备端员工/记录增量同步	limit 默认 100，最大 500
设备端员工分页查询	pageSize 默认 20，最大 100
导出文件下载	文件生成后 1 小时内有效

二、设备端 REST API

设备内置 Ktor 服务器，提供以下 REST 接口供客户系统调用。

2.1 认证方式

除 GET /health 和 POST /api/auth/login 外，所有接口需要在请求头中携带 Token：

```
Authorization: Bearer {token}
```

Token 通过登录接口获取，有效期 24 小时。

2.2 通用响应格式

成功响应

```
{
  "code": 0,
  "message": "success",
  "data": { }
}
```

错误响应

```
{
  "code": 1001,
  "message": "错误描述",
  "data": null
}
```

错误码说明

错误码	说明
0	成功
1001	未授权（Token 无效或过期）
1002	禁止访问
1003	资源不存在
1004	参数错误
1005	服务器内部错误
1006	Token 已过期
2005	用户名已存在

错误码	说明
2006	数据验证失败
2009	密码不能为空
2010	无效的角色
2011	无效的状态

2.3 接口列表

序号	接口	方法	认证	功能
1	/health	GET	否	健康检查
2	/api/auth/login	POST	否	登录获取 Token
3	/api/employees	GET	是	获取员工列表
4	/api/employees/sync	GET	是	员工增量同步
5	/api/employees/import	POST	是	批量导入员工
6	/api/employees/upload-photo	POST	是	上传员工人脸照片
7	/api/employees/upload-cert-photo	POST	是	上传员工健康证照片
8	/api/employees/{employeeId}	DELETE	是	删除员工
9	/api/employees/clear-all	DELETE	是	清空所有员工
10	/api/records	GET	是	查询晨检记录列表
11	/api/records/{id}	GET	是	查询单条记录详情
12	/api/records/sync	GET	是	增量同步记录
13	/api/records/statistics	GET	是	统计汇总
14	/api/records/export	POST	是	导出 CSV 文件
15	/api/images/{filename}	GET	是	下载检测图片
16	/api/employee-images/{filename}	GET	是	下载员工照片
17	/api/downloads/{filename}	GET	是	下载导出文件
18	/api/users/sync	POST	是	账号信息同步

2.4 接口详情

1. 健康检查

检查服务是否正常运行，无需认证。

```
GET /health
```

响应

```
{
  "code": 0,
  "message": "success",
  "data": {
    "status": "ok",
    "timestamp": 1716259200000
  }
}
```

2. 登录

获取访问 Token。

```
POST /api/auth/login
Content-Type: application/json
```

```
{
  "username": "admin",
  "password": "123456"
}
```

响应

```
{
  "code": 0,
  "message": "success",
  "data": {
    "token": "eyJhbGciOiJIUzI1NiIs...\"",
    "expiresIn": 86400,
    "tokenType": "Bearer",
    "user": {
      "id": 1,
      "username": "admin",
      "name": "管理员"
    }
  }
}
```

3. 获取员工列表

```
GET /api/employees?page=1&pageSize=20&employeeId=001&isActive=true
Authorization: Bearer {token}
```

参数

参数	类型	必填	说明
page	int	否	页码，默认 1
pageSize	int	否	每页条数，默认 20，最大 100
employeeId	string	否	员工工号，精确匹配
isActive	boolean	否	是否在职

响应

```
{
  "code": 0,
  "message": "success",
  "data": {
    "list": [
      {
        "id": 1,
        "name": "张三",
        "employeeId": "001",
        "idCardNumber": "110101199001011234",
        "healthCertStatus": "VALID",
        "healthCertStartDate": 1704067200000,
        "healthCertEndDate": 1735689600000,
        "faceImage": "/api/employee-images/face_001.jpg",
        "healthCertImage": "/api/employee-images/cert_001.jpg",
        "isActive": true,
        "createdAt": 1704067200000
      }
    ],
    "pagination": {
      "page": 1,
      "pageSize": 20,
      "total": 10,
      "totalPages": 1
    }
  }
}
```

健康证状态说明

状态	说明
VALID	有效
EXPIRING_SOON	即将过期（7天内）
EXPIRED	已过期
NOT_FOUND	未找到

4. 员工增量同步

用于避免重复拉取，只获取新增员工。

```
GET /api/employees/sync?lastEmployeeId=0&limit=100
Authorization: Bearer {token}
```

参数

参数	类型	必填	说明
lastEmployeeId	long	否	上次同步的最后员工 ID，默认 0
limit	int	否	拉取条数，默认 100，最大 500

响应

```
{
  "code": 0,
  "message": "success",
  "data": {
    "list": [...],
    "hasMore": true,
    "lastRecordId": 10,
    "syncTime": 1706745600000
  }
}
```

兼容说明：员工增量接口复用通用同步响应模型，因此员工游标字段当前仍命名为 `lastRecordId`。该值实际表示本批最后一名员工的内部数字 ID，下一次请求应将其作为 `lastEmployeeId` 传入。

5. 批量导入员工

支持增量导入（跳过已存在）或覆盖导入（更新已存在）。支持同时传入人脸照片和健康证照片的 Base64 编码。

```
POST /api/employees/import
Authorization: Bearer {token}
Content-Type: application/json

{
  "employees": [
    {
      "name": "张三",
      "employeeId": "001",
      "idCardNumber": "110101199001011234",
      "phone": "13800138000",
      "position": "厨师",
      "department": "后厨",
      "healthCertCode": "HC2024001",
      "faceImageBase64": "/9j/4AAQSkZJRg...",
      "healthCertImageBase64": "/9j/4AAQSkZJRg...",
      "healthCertStartDate": 1704067200000,
      "healthCertEndDate": 1735689600000,
      "isActive": true
    }
  ],
  "incremental": true
}
```

参数

参数	类型	必填	说明
employees	array	是	员工列表，最大 100 条
incremental	boolean	否	是否增量导入，默认 true
name	string	是	员工姓名
employeeId	string	是	员工工号（唯一）
idCardNumber	string	否	身份证号
phone	string	否	电话
position	string	否	职位
department	string	否	部门
healthCertCode	string	否	健康证编号
faceImageBase64	string	否	人脸照片 Base64
healthCertImageBase64	string	否	健康证照片 Base64
healthCertStartDate	long	否	健康证开始日期（时间戳）
healthCertEndDate	long	否	健康证结束日期（时间戳）
isActive	boolean	否	是否在职，默认 true

响应

```
{
  "code": 0,
  "message": "success",
  "data": {
    "total": 3,
    "success": 2,
    "failed": 0,
    "details": [
      {
        "employeeId": "001",
        "status": "success",
        "message": "导入成功（含人脸特征）",
        "userId": 1
      },
      {
        "employeeId": "002",
        "status": "skipped",
        "message": "员工工号已存在，跳过",
        "userId": 2
      },
      {
        "employeeId": "003",
        "status": "updated",
        "message": "更新成功",
        "userId": 3
      }
    ]
  }
}
```

状态说明

status	说明
success	新导入成功
skipped	增量模式下已存在，跳过
updated	非增量模式下更新成功
failed	导入失败

6. 上传员工人脸照片

为员工单独上传人脸照片，自动检测人脸并提取特征。员工工号必须已存在。

```
POST /api/employees/upload-photo
Authorization: Bearer {token}
```



```
Content-Type: application/json

{
  "fileName": "face_001.jpg",
  "imageBase64": "/9j/4AAQSkZJRgABAQAAQ..."
}
```

参数

参数	类型	必填	说明
fileName	string	是	文件名，格式：face_{工号}.jpg
imageBase64	string	是	人脸照片 Base64 编码

响应

```
{
  "code": 0,
  "message": "success",
  "data": {
    "employeeId": "001",
    "userId": 1,
    "faceImagePath": "face_emp_1716259200000_1.jpg",
    "message": "上传成功，已提取人脸特征"
  }
}
```

7. 上传员工健康证照片

```
POST /api/employees/upload-cert-photo
Authorization: Bearer {token}
Content-Type: application/json

{
  "fileName": "cert_001.jpg",
  "imageBase64": "/9j/4AAQSkZJRgABAQAAQ..."
}
```

参数

参数	类型	必填	说明
fileName	string	是	文件名，格式：cert_{工号}.jpg/jpeg/png
imageBase64	string	是	健康证照片 Base64 编码

8. 删除员工

```
DELETE /api/employees/{employeeId}
Authorization: Bearer {token}
```

响应

```
{
  "code": 0,
  "message": "success",
  "data": {
    "employeeId": "001",
    "deleted": true
  }
}
```

9. 清空所有员工

```
DELETE /api/employees/clear-all
Authorization: Bearer {token}
```

响应

```
{
  "code": 0,
  "message": "success",
  "data": {
    "deleted": true,
    "message": "所有员工已清空"
  }
}
```

此操作不可恢复，请谨慎使用。

10. 查询晨检记录列表

```
GET /api/records?startDate=2026-01-01&endDate=2026-01-31&page=1&pageSize=20
Authorization: Bearer {token}
```

参数

参数	类型	必填	说明
startDate	string	是	开始日期，格式：yyyy-MM-dd
endDate	string	是	结束日期，格式：yyyy-MM-dd
employeeId	string	否	员工工号，精确匹配
isPassed	boolean	否	是否通过晨检
isTempNormal	boolean	否	体温是否正常
isHandNormal	boolean	否	手部是否正常
includeImages	boolean	否	是否包含图片 URL，默认 false
page	int	否	页码，默认 1
pageSize	int	否	每页条数，默认 20

响应

```
{
  "code": 0,
  "message": "success",
  "data": {
    "list": [
      {
        "id": 1001,
        "userId": 1,
        "userName": "张三",
        "employeeId": "001",
        "checkTime": 1706745600000,
        "temperature": 36.5,
        "isTempNormal": true,
        "isHandNormal": true,
        "isPassed": true,
        "handStatus": "NORMAL",
        "healthCertStatus": "VALID",
        "symptomFlags": "",
        "remark": "",
        "images": {
          "face": "/api/images/face_20260101_001.jpg",
          "palm": "/api/images/palm_20260101_001.jpg",
          "back": "/api/images/back_20260101_001.jpg"
        }
      }
    ],
    "pagination": {
      "page": 1,
      "pageSize": 20,
      "total": 150,
      "totalPages": 8
    }
  }
}
```

```
}  
}
```

11. 查询单条记录详情

```
GET /api/records/{id}?includeImages=true  
Authorization: Bearer {token}
```

参数	类型	必填	说明
id	long	是	记录 ID（路径参数）
includeImages	boolean	否	是否包含图片 URL，默认 true

12. 增量同步记录

```
GET /api/records/sync?lastRecordId=1000&limit=100  
Authorization: Bearer {token}
```

参数	类型	必填	说明
lastRecordId	long	是	上次同步的最后记录 ID
limit	int	否	拉取条数，默认 100，最大 500

响应

```
{  
  "code": 0,  
  "message": "success",  
  "data": {  
    "list": [...],  
    "hasMore": true,  
    "lastRecordId": 1100,  
    "syncTime": 1706745600000  
  }  
}
```

使用流程

- 1. 首次同步：lastRecordId=0
- 2. 保存返回的 lastRecordId
- 3. 下次同步使用上次返回的值

4. 若 `hasMore=true`，继续拉取

13. 统计汇总

```
GET /api/records/statistics?startDate=2026-01-01&endDate=2026-01-31
Authorization: Bearer {token}
```

参数	类型	必填	说明
startDate	string	是	开始日期， yyyy-MM-dd
endDate	string	是	结束日期， yyyy-MM-dd

响应

```
{
  "code": 0,
  "message": "success",
  "data": {
    "totalCheck": 150,
    "passed": 145,
    "failed": 5,
    "tempAbnormal": 2,
    "handAbnormal": 3,
    "dailyStats": [
      {
        "date": "2026-01-01",
        "total": 50,
        "passed": 48,
        "failed": 2,
        "tempAbnormal": 1,
        "handAbnormal": 1
      }
    ]
  }
}
```

14. 导出记录

导出指定日期范围的记录为 CSV 文件。

```
POST /api/records/export
Authorization: Bearer {token}
Content-Type: application/json

{
```

```
"startDate": "2026-01-01",
"endDate": "2026-01-31",
"format": "csv",
"employeeId": "001"
}
```

参数	类型	必填	说明
startDate	string	是	开始日期
endDate	string	是	结束日期
format	string	否	当前仅支持 csv ，默认 csv
employeeId	string	否	预留字段；当前版本导出时尚未应用员工筛选

响应

```
{
  "code": 0,
  "message": "success",
  "data": {
    "downloadUrl": "/api/downloads/records_2026-01-01_2026-01-31.csv",
    "expiresAt": 1706749200000
  }
}
```

文件 1 小时有效期，过期自动删除。

15~17. 文件下载

接口	说明
GET /api/images/{filename}	下载检测图片（人脸/手掌/手背）
GET /api/employee-images/{filename}	下载员工照片（人脸/健康证）
GET /api/downloads/{filename}	下载导出文件

成功返回文件二进制数据，失败返回 JSON 错误。

文件名会做安全检查，不支持 `../\` 等路径穿越字符。

18. 账号信息同步

同步员工登录账号（非员工档案，而是可登录设备的账号）。

```
POST /api/users/sync
Authorization: Bearer {token}
Content-Type: application/json

{
  "action": "create",
  "users": [
    {
      "username": "001",
      "password": "123456",
      "passwordType": "plain",
      "employeeId": "001",
      "role": "employee",
      "status": "active"
    }
  ]
}
```

参数	类型	必填	说明
action	string	是	操作：create / update / delete / sync
users	array	是	用户列表，最大 100 条
username	string	是	登录用户名
password	string	条件必填	create / sync 时建议必填；update 不传则保留原密码，delete 不需要
passwordType	string	否	密码类型标记：plain / md5 / bcrypt，默认 plain
employeeId	string	否	关联员工工号
role	string	否	角色：employee / admin，默认 employee
status	string	否	状态：active / disabled，默认 active

action 说明

action	行为
create	新增（已存在则失败）
update	更新（不存在则失败）
delete	删除（不存在则失败）
sync	全量同步（先清空所有账号，再新增）

响应

```
{
  "code": 0,
```

```
"message": "success",
"data": {
  "received": 2,
  "success": 2,
  "failed": 0,
  "details": [
    {
      "username": "001",
      "status": "success",
      "message": "同步成功"
    }
  ]
}
```

三、平台接入接口

以下接口需要**客户平台实现**，供晨检仪设备主动调用。

3.1 认证方式

所有平台接入接口使用 **api-key** 请求头认证：

头字段	说明
api-key	客户在设备「设置 → 平台接入」中配置的 API Key

3.2 接口列表

序号	接口	方法	功能
1	/api/device/refresh	POST	设备心跳
2	/api/device/morning-check/upload	POST	上报晨检记录
3	/api/device/employees/changes	POST	上传本地员工变更
4	/api/device/employees/changes	GET	拉取平台员工增量变更
5	/api/device/employees/{employeeId}	GET	查询单个员工最新状态
6	/api/device/employees/snapshot	GET	拉取完整员工快照
7	/api/device/employees/images/{fileId}	GET	下载员工图片

3.3 接口详情

1. 设备心跳

设备定期向平台发送心跳，证明设备在线。心跳间隔可在设备设置中配置（默认 30 秒，范围 10~300 秒）。


```
POST /api/device/refresh
api-key: {客户分配的API Key}
```

请求体：无（空 body）

响应示例

```
{
  "code": 200,
  "message": "success",
  "data": {
    "device_id": "DEVICE001",
    "status": "online",
    "last_heartbeat": "2026-07-28T08:00:00Z"
  }
}
```

响应字段说明

字段	类型	说明
code	int	状态码，200 表示成功
message	string	响应消息
data.device_id	string	设备唯一标识
data.status	string	设备状态
data.last_heartbeat	string	最后心跳时间

2. 上报晨检记录

每次晨检完成后，App 自动将记录上报到平台。图片以 Base64（JPEG）编码传输。

```
POST /api/device/morning-check/upload
Content-Type: application/json
api-key: {客户分配的API Key}

{
  "device_id": "DEVICE001",
  "timestamp": 1716259200000,
  "employees": [
    {
      "id": "E001",
      "name": "张三",
      "temperature": 36.5,
      "photo": "/9j/4AAQSkZJRgABAQAAQ...",
    }
  ]
}
```

```
        "hand_palm_photo": "/9j/4AAQSkZJRgABAQAAAQ...",
        "hand_back_photo": "/9j/4AAQSkZJRgABAQAAAQ..."
    }
}
}
```

请求字段说明

字段	类型	必填	说明
device_id	string	是	设备唯一标识
timestamp	long	是	检测时间戳（毫秒）
employees	array	是	员工检测数据列表（当前每次 1 人）
employees[].id	string	是	员工工号
employees[].name	string	是	员工姓名
employees[].temperature	float	是	体温（摄氏度）
employees[].photo	string	否	人脸照片 Base64（JPEG）
employees[].hand_palm_photo	string	否	手掌照片 Base64（JPEG）
employees[].hand_back_photo	string	否	手背照片 Base64（JPEG）

响应示例

```
{
  "code": 200,
  "message": "success",
  "data": {
    "recordId": "REC001",
    "processTime": 120,
    "warnings": []
  }
}
```

响应字段说明

字段	类型	说明
code	int	状态码，200 表示成功
message	string	响应消息
data.recordId	string	平台生成的记录 ID
data.processTime	int	处理耗时（毫秒）
data.warnings	array	警告信息列表

注意：

- 若平台地址或 API Key 未配置，上报会自动跳过
- 上报失败的数据会保留在本地，网络恢复后自动重试
- 图片 Base64 字符串可能较长，服务端需做好接收准备

3. 上传员工变更

设备将本地新增、修改或删除的员工操作批量上传到平台。平台必须使用 `operation_id` 做幂等处理，并通过版本号检测并发冲突。

```
POST /api/device/employees/changes
Content-Type: application/json
api-key: {客户分配的API Key}
```

请求示例

```
{
  "device_id": "DEVICE001",
  "batch_id": "6b152909-7bb8-4d9f-8e40-624b66998b13",
  "timestamp": 1785168000000,
  "operations": [
    {
      "operation_id": "bd43ca64-ad6f-42c3-88cd-edf70985df89",
      "type": "UPSERT",
      "employee_id": "E001",
      "expected_version": 3,
      "employee": {
        "name": "张三",
        "id_card_number": "110101199001011234",
        "phone": "13800138000",
        "position": "厨师",
        "department": "后厨",
        "status": "ACTIVE",
        "face_image": {
          "action": "REPLACE",
          "mime_type": "image/jpeg",
          "sha256": "图片原始字节的SHA-256十六进制值",
          "base64": "图片原始字节的Base64，不含data:前缀"
        }
      },
      "health_certificate": {
        "code": "HC2026001",
        "start_date": "2026-01-01",
        "end_date": "2027-01-01",
        "status": "VALID",
        "image": {
          "action": "KEEP"
        }
      }
    }
  ]
}
```

```
    }
  },
  {
    "operation_id": "75ba6f64-06ce-42cc-b227-90b65ddd5683",
    "type": "DELETE",
    "employee_id": "E002",
    "expected_version": 5
  }
]
```

请求字段说明

字段	类型	必填	说明
device_id	string	是	设备唯一标识
batch_id	string	是	批次 UUID，用于批次追踪
timestamp	long	是	请求生成时间戳（毫秒）
operations	array	是	员工变更操作列表
operations[].operation_id	string	是	操作 UUID；平台必须据此保证幂等
operations[].type	string	是	UPSERT 或 DELETE
operations[].employee_id	string	是	员工工号
operations[].expected_version	long/null	否	设备已知的平台版本；首次创建可为 null
operations[].employee	object	条件必填	UPSERT 必填，DELETE 不传

员工对象字段

字段	类型	必填	说明
name	string	是	姓名
id_card_number	string/null	否	身份证号
phone	string/null	否	电话
position	string/null	否	职位
department	string/null	否	部门
status	string	是	ACTIVE 或 DISABLED
face_image	object	是	人脸图片操作
health_certificate	object/null	否	健康证对象

健康证对象字段

字段	类型	必填	说明
code	string/null	否	健康证编号
start_date	string	是	有效期开始日期，格式 YYYY-MM-DD
end_date	string	是	有效期结束日期，格式 YYYY-MM-DD
status	string	是	VALID、EXPIRED 或 REVOKED
image	object	是	健康证图片操作，结构同下方图片操作字段

图片操作字段

字段	类型	必填	说明
action	string	是	KEEP 保留、CLEAR 清除、REPLACE 替换
mime_type	string/null	REPLACE 时是	image/jpeg 或 image/png
sha256	string/null	REPLACE 时是	图片原始二进制的 SHA-256
base64	string/null	REPLACE 时是	图片原始二进制 Base64，不含 Data URL 前缀和换行

成功响应示例

```
{
  "code": 200,
  "message": "success",
  "data": {
    "batch_id": "6b152909-7bb8-4d9f-8e40-624b66998b13",
    "accepted": 1,
    "duplicates": 0,
    "conflicts": 1,
    "rejected": 0,
    "server_cursor": 105,
    "results": [
      {
        "operation_id": "bd43ca64-ad6f-42c3-88cd-edf70985df89",
        "employee_id": "E001",
        "status": "APPLIED",
        "employee_version": 4,
        "cursor": 104
      },
      {
        "operation_id": "75ba6f64-06ce-42cc-b227-90b65ddd5683",
        "employee_id": "E002",
        "status": "CONFLICT",
        "employee_version": 7,
        "cursor": 105,
        "error_code": 409,
        "message": "version conflict"
      }
    ]
  }
}
```

```
    }  
  }  
}
```

`results[].status` 可取 `APPLIED`、`DUPLICATE`、`CONFLICT`、`REJECTED`。发生冲突时，设备会再查询该员工的最新状态并执行冲突处理。

4. 拉取员工增量变更

设备按全局游标拉取平台侧员工变化。平台应按 `cursor` 升序返回稳定结果。

```
GET /api/device/employees/changes?after_cursor=100&limit=100  
api-key: {客户分配的API Key}
```

参数	类型	必填	说明
after_cursor	long	是	上次成功应用的游标；首次同步传 0
limit	int	否	本批最大条数，设备默认传 100

响应示例

```
{  
  "code": 200,  
  "message": "success",  
  "data": {  
    "changes": [  
      {  
        "cursor": 101,  
        "type": "UPSERT",  
        "employee_id": "E001",  
        "employee_version": 4,  
        "origin_device_id": "DEVICE002",  
        "employee": {  
          "employee_id": "E001",  
          "name": "张三",  
          "id_card_number": "110101199001011234",  
          "phone": "13800138000",  
          "position": "厨师",  
          "department": "后厨",  
          "status": "ACTIVE",  
          "face_image": {  
            "file_id": "face-001-v4",  
            "sha256": "图片SHA-256",  
            "download_url": "/api/device/employees/images/face-001-v4"  
          },  
          "health_certificate": null,  
          "version": 4,  
          "updated_at": 1785168000000  
        }  
      }  
    ]  
  }  
}
```

```
    }
  },
  {
    "cursor": 102,
    "type": "DELETE",
    "employee_id": "E002",
    "employee_version": 8,
    "origin_device_id": "DEVICE002",
    "employee": null
  }
],
"next_cursor": 102,
"has_more": false,
"server_time": 1785168000000
}
```

设备仅在整批变化成功落库后推进本地游标；`has_more=true`时会继续使用 `next_cursor` 拉取下一批。

5. 查询单个员工最新状态

设备在版本冲突或定向修复时查询单个员工。

```
GET /api/device/employees/{employeeId}
api-key: {客户分配的API Key}
```

响应示例

```
{
  "code": 200,
  "message": "success",
  "data": {
    "employee_id": "E001",
    "deleted": false,
    "version": 4,
    "employee": {
      "employee_id": "E001",
      "name": "张三",
      "status": "ACTIVE",
      "face_image": null,
      "health_certificate": null,
      "version": 4,
      "updated_at": 1785168000000
    }
  }
}
```

员工已删除时，平台返回 `deleted=true`，且 `employee` 为 `null`。

6. 拉取完整员工快照

首次同步、游标失效或本地数据修复时，设备拉取完整快照。

```
GET /api/device/employees/snapshot
api-key: {客户分配的API Key}
```

响应示例

```
{
  "code": 200,
  "message": "success",
  "data": {
    "employees": [],
    "total": 0,
    "cursor": 105,
    "server_time": 1785168000000
  }
}
```

`employees` 中的对象结构与“拉取员工增量变更”中的 `employee` 一致。`cursor` 表示快照对应的平台最新游标，设备应用快照后从该游标继续增量同步。

7. 下载员工图片

设备当前使用平台返回的 `file_id`，通过标准图片接口下载员工人脸、健康证图片。`download_url` 仍为必返字段，可返回同一路径。

```
GET /api/device/employees/images/{fileId}
api-key: {客户分配的API Key}
```

成功响应：图片原始二进制数据，`Content-Type` 应为 `image/jpeg` 或 `image/png`。

失败响应：返回非 2xx HTTP 状态，或使用统一 JSON 错误结构。不要在 HTTP 2xx 响应中返回 JSON 错误，否则设备会将其识别为下载失败。

3.4 平台响应约定

除图片下载接口外，平台接口应返回以下包装结构：


```
{
  "code": 200,
  "message": "success",
  "data": {}
}
```

为兼容部分平台实现，设备也会读取 `msg` 作为错误消息；新接入统一使用 `message`。设备同时要求 HTTP 状态为 2xx 且业务 `code=200`，否则按失败处理。

3.5 员工同步实现要求

1. `operation_id` 必须全局幂等；重复提交返回 `DUPLICATE`，不得重复产生版本或游标。
2. 每次成功的员工状态变化必须生成单调递增的 `cursor` 和员工 `version`。
3. `expected_version` 不匹配时返回 `CONFLICT`，并返回当前 `employee_version`。
4. `DELETE` 变化的 `employee` 必须为 `null`，但仍需保留可供增量拉取的删除事件。
5. 图片 `download_url` 可以是相对路径；推荐同时保证 `file_id` 可通过标准图片接口下载。
6. 日期字段 `start_date`、`end_date` 使用 `YYYY-MM-DD`；`updated_at`、`server_time` 使用毫秒时间戳。
7. 图片 Base64 不携带 `data:image/...;base64`，前缀，不插入换行。

四、调用示例

JavaScript

```
// 1. 登录获取 Token
const loginRes = await fetch('http://192.168.1.100:8080/api/auth/login', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ username: 'admin', password: '123456' })
});
const { data: { token } } = await loginRes.json();

// 2. 查询晨检记录
const recordsRes = await fetch(
  'http://192.168.1.100:8080/api/records?startDate=2026-01-01&endDate=2026-01-31',
  { headers: { 'Authorization': `Bearer ${token}` } }
);
const { data } = await recordsRes.json();
console.log(data.list);
```

Python

```
import requests
```

```
# 1. 登录
login_res = requests.post('http://192.168.1.100:8080/api/auth/login', json=
{
    'username': 'admin',
    'password': '123456'
})
token = login_res.json()['data']['token']

# 2. 查询记录
records_res = requests.get(
    'http://192.168.1.100:8080/api/records?startDate=2026-01-
01&endDate=2026-01-31',
    headers={'Authorization': f'Bearer {token}'})
data = records_res.json()['data']
print(data['list'])
```

cURL

```
# 登录
curl -X POST http://192.168.1.100:8080/api/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"admin","password":"123456"}'

# 查询记录 (替换 {token})
curl "http://192.168.1.100:8080/api/records?startDate=2026-01-
01&endDate=2026-01-31" \
-H "Authorization: Bearer {token}"
```

五、注意事项

1. **设备 IP**：实际使用时替换为设备的真实 IP 地址
2. **网络要求**：确保客户系统能访问设备的 8080 端口
3. **Token 有效期**：Token 有效期为 24 小时，过期后需重新登录
4. **时间戳**：所有时间戳单位为毫秒（Java 标准）
5. **分页建议**：单次查询建议不超过 100 条，大数据量请使用增量同步
6. **图片下载**：记录图片和员工图片需在请求时携带有效 Token；导出文件额外受 1 小时有效期限限制
7. **账号同步**：同步账号后，需要在设备上**重新登录**使新账号生效
8. **全量同步**：**action=sync** 会清空所有现有账号，请谨慎使用
9. **平台接入配置**：使用平台接入接口前，需在设备「设置 → 平台接入」中配置平台地址和 API Key
10. **心跳间隔**：设备心跳默认 30 秒一次，可在设置中调整（10~300 秒）
11. **上报重试**：网络异常导致上报失败时，数据会保留在本地，恢复后自动补发
12. **员工同步**：平台必须实现操作幂等、版本冲突和单调游标，否则多设备同步可能产生重复或覆盖
13. **安全建议**：设备端 API 当前为明文 HTTP，仅建议部署在可信局域网；跨网访问应通过 HTTPS 反向代理或 VPN

六、更新日志

版本	日期	说明
2.0	2026-07-28	补齐员工双向同步的 5 个平台接口 新增员工、图片、版本号、游标和幂等规则 补充接口方向、请求限制与平台统一响应约定 修正员工增量游标字段、导出参数和文件有效期说明
1.8	2026-06-09	新增平台接入接口章节（设备心跳、晨检记录上报） 新增 api-key 认证方式说明 修正 Token 有效期为 24 小时 移除旧兼容接口，专注对外标准接口
1.7	2026-05-22	新增健康检查接口、员工照片上传接口、导出下载接口
1.6	2026-03-06	新增删除/清空员工接口，导入支持增量/覆盖模式
1.5	2026-03-05	新增员工照片上传接口
1.4	2026-03-05	新增批量导入员工接口
1.3	2026-03-05	员工接口新增人脸/健康证照片字段
1.2	2026-03-05	新增员工信息接口
1.1	2026-03-03	新增账号同步接口
1.0	2026-03-03	初始版本